

## 117 - How tokens are generated and validated

# Token Generation and Validation

- Email Confirmation Token : Parts 113 & 114
- Password Reset Token : Parts 115 & 116

### To generate Email Confirmation Token

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

### To generate Password Reset Token

```
var token = await userManager.GeneratePasswordResetTokenAsync(user);
```

```
var token = await userManager.GenerateEmailConfirmationTokenAsync(user);
```

```
var token = await userManager.GeneratePasswordResetTokenAsync(user);
```

Both these methods, [GenerateEmailConfirmationTokenAsync\(\)](#)

and [GeneratePasswordResetTokenAsync\(\)](#)

internally calls [GenerateUserTokenAsync\(\)](#) method.

<https://github.com/dotnet/aspnetcore/blob/release/5.0/src/Identity/Extensions.Core/src/UserManager.cs>

```
namespace Microsoft.AspNetCore.Identity
{
    /// <summary>
    /// Provides the APIs for managing user in a persistence store.
    /// </summary>
    /// <typeparam name="TUser">The type encapsulating a user.</typeparam>
    public class UserManager<TUser> : IDisposable where TUser : class
    {
        /// <summary>
        /// The data protection purpose used for the reset password related methods.
        /// </summary>
        public const string ResetPasswordTokenPurpose = "ResetPassword";

        /// <summary>
        /// The data protection purpose used for the change phone number methods.
        /// </summary>
        public const string ChangePhoneNumberTokenPurpose = "ChangePhoneNumber";

        /// <summary>
        /// The data protection purpose used for the email confirmation related methods.
        /// </summary>
        public const string ConfirmEmailTokenPurpose = "EmailConfirmation";
    }
}
```

# Token Generation and Validation

```
public virtual Task<string> GenerateEmailConfirmationTokenAsync(TUser user)
{
    ThrowIfDisposed();
    return GenerateUserTokenAsync(user,
        Options.Tokens.EmailConfirmationTokenProvider, ConfirmEmailTokenPurpose);
}

public virtual Task<string> GeneratePasswordResetTokenAsync(TUser user)
{
    ThrowIfDisposed();
    return GenerateUserTokenAsync(user,
        Options.Tokens.PasswordResetTokenProvider, ResetPasswordTokenPurpose);
}
```

```
public virtual Task<string>
    GenerateUserTokenAsync(TUser user, string tokenProvider, string purpose)
{
    // Method Implementation
}
```

`GenerateUserTokenAsync()` has 3 parameters.

- The `user` the token will be for
- The `token provider` which will actually generate the token
- The token purpose - For example, password reset or email confirmation. Token generated for a given purpose must be used only for that purpose. For example, an **email confirmation token cannot be used to reset a password**.

```
public virtual Task<string>
    GenerateEmailConfirmationTokenAsync(TUser user)
{
    ThrowIfDisposed();
    return GenerateUserTokenAsync(user,
        Options.Tokens.EmailConfirmationTokenProvider, ConfirmEmailTokenPurpose);
}

public virtual Task<string>
    GeneratePasswordResetTokenAsync(TUser user)
{
    ThrowIfDisposed();
    return GenerateUserTokenAsync(user,
        Options.Tokens.PasswordResetTokenProvider, ResetPasswordTokenPurpose);
}
```

Both [Email Confirmation Tokens](#)  
and [Password Reset Tokens](#)  
are generated by the built-in [DataProtectorTokenProvider](#).

# DataProtectorTokenProvider Class

- **GenerateAsync()** : Generates Token
- **ValidateAsync()** : Validates Token

**GenerateAsync()** method generates the token and

**ValidateAsync()** method validates the token

The source code of this class is on the following github page

<https://github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProvider.cs>

## DataProtectorTokenProvider GenerateAsync()

As you can see from the code, the generated token contains

- Token Creation Time
- User ID
- Token Purpose
- Security Stamp

All this data is then encrypted and then base 64 encoded so it can sent over the network. ASP.NET Core uses **Data Protection API** (in short **DP API**) for encryption. Later in this video series, we will discuss how to use DP API and encrypt our application data like query strings for example.



```
public virtual async Task<string> GenerateAsync(string purpose, UserManager<TUser> manager, TUser user)
{
    if (user == null)
    {
        throw new ArgumentNullException(nameof(user));
    }
    var ms = new MemoryStream();
    var userId = await manager.GetUserIdAsync(user);
    using (var writer = ms.CreateWriter())
    {
        writer.Write(DateTimeOffset.UtcNow); creation time
        writer.Write(userId);
        writer.Write(purpose ?? "");
        string stamp = null;
        if (manager.SupportsUserSecurityStamp)
        {
            stamp = await manager.GetSecurityStampAsync(user);
        }
        writer.Write(stamp ?? ""); using DP API - data protection API encryption and decryption
    }
    var protectedBytes = Protector.Protect(ms.ToArray());
    return Convert.ToBase64String(protectedBytes);
} base 64 encoded to be safely send over the wire
```

```

public virtual async Task<string> GenerateAsync(string purpose,
    UserManager<TUser> manager, TUser user)
{
    if (user == null)
    {
        throw new ArgumentNullException(nameof(user));
    }
    var ms = new MemoryStream();
    var userId = await manager.GetUserIdAsync(user);
    using (var writer = ms.CreateWriter())
    {
        writer.Write(DateTimeOffset.UtcNow);
        writer.Write(userId);
        writer.Write(purpose ?? "");
        string stamp = null;
        if (manager.SupportsUserSecurityStamp)
        {
            stamp = await manager.GetSecurityStampAsync(user);
        }
        writer.Write(stamp ?? "");
    }
    var protectedBytes = Protector.Protect(ms.ToArray());
    return Convert.ToBase64String(protectedBytes);
}

```

## DataProtectorTokenProvider ValidateAsync()

As the name implies `ValidateAsync()` method validates the token. First it is decoded from Base 64 and then decrypted. Decryption is done by the Data Protection API (DP API).

The token `creation time` is read from the token. To this, the the token life span is added. If this computed DateTime is less than current UTC DateTime, the token has expired. So the method returns false. **Tokens cannot be used after they have expired.** The default token lifespan is one day. We can change this to meet our application requirements. We will discuss how to do this in our next video.

The `User ID` in the token is compared against the User ID the token is being used. If the **User IDs are not equal**, the method return false invalidating the token. A token generated for a given user must only be used by that user.

The purpose from the token is read to make sure it is used for the intended purpose. **Token generated for a purpose must be used only for that purpose.** If we try to use it for a different purpose, token validation fails. For example, if a token is generated for a given user for the purpose of resetting password, it can be used only by that specific user for resetting his password. If we try to use it for a different user or purpose, token validation fails. Simply put, Password Reset Token cannot be used to confirm an email address.

The security stamp in the token is compared with the user current security stamp in the database. If they are different, the token validation fails.

```

public virtual async Task<bool> ValidateAsync(string purpose,

```

```

        string token, UserManager<TUser> manager, TUser user)
    {
        try
        {
            var unprotectedData =
Protector.Unprotect(Convert.FromBase64String(token));
            var ms = new MemoryStream(unprotectedData);
            using (var reader = ms.CreateReader())
            {
                var creationTime = reader.ReadDateTimeOffset();
                var expirationTime = creationTime + Options.TokenLifespan;
                if (expirationTime < DateTimeOffset.UtcNow)
                {
                    return false;
                }

                var userId = reader.ReadString();
                var actualUserId = await manager.GetUserIdAsync(user);
                if (userId != actualUserId)
                {
                    return false;
                }
                var purp = reader.ReadString();
                if (!string.Equals(purp, purpose))
                {
                    return false;
                }
                var stamp = reader.ReadString();
                if (reader.PeekChar() != -1)
                {
                    return false;
                }

                if (manager.SupportsUserSecurityStamp)
                {
                    return stamp == await manager.GetSecurityStampAsync(user);
                }
                return stamp == "";
            }
        }
        // ReSharper disable once EmptyGeneralCatchClause
        catch
        {
            // Do not leak exception
        }
        return false;
    }
}

```

# DataProtectorTokenProvider Class

- **GenerateAsync()** : Generates Token
- **ValidateAsync()** : Validates Token

## Generated Token Contains



## Token Validation Fails

**Tampered and cannot be decrypted**

**Expired**

**Used against a different User ID**

**Used for a different purpose**

**Security stamp is different**